Synthetic RGB-D of a manufacturing cell with a RealSense D435 depth model

Ravi Kola

November 2026

1 Overview

A humanoid robot in a manufacturing cell picks an angle bracket from a table next to an aluminium profile, using its head depth camera. We take the Unitree G1 as the robot and its head camera, an Intel RealSense D435, as the reference sensor. The D435 is an active infrared stereo camera, documented in Intel's datasheet [1] and depth tuning guide [2], and its stereo failure modes are the ones small metal features are expected to provoke. The work centres on the camera's depth model and its effects on the downstream perception and robot tasks.

We render labelled RGB-D frames of the cell in simulation from scene description, corrupt the ideal depth with an analytical D435 model built from published numbers, and run a depth-only pipeline on it. The task is to detect the bracket's top hole in 6D, fit the bracket's CAD model, and grasp the bracket with the jaws approaching along the hole axis (Fig. 1). On 30 held-out test scenes, all of which succeed with ideal depth, the full camera model lowered hole detection from 100 % to 54 %. The CAD fit still found the bracket in 99 % of the cases, but displaced by about 11 mm, and the grasp success fell to 69 %.

The results suggest a few takeaways. Cues only a few pixels wide, such as the hole, appear to suffer first, while a fit over the whole surface tolerated the random noise but not the systematic bias. The grasp result reflects the planner's sensitivity as well as the camera's. A brief check against real D435 depth suggests the model is of the right kind and size, though it is not calibrated for this cell. Finally, the report outlines what this may mean for models trained on synthetic depth.

Code, data and reproduction steps: `https://github.com/ravikola02/synthetic_rgbd_task`.

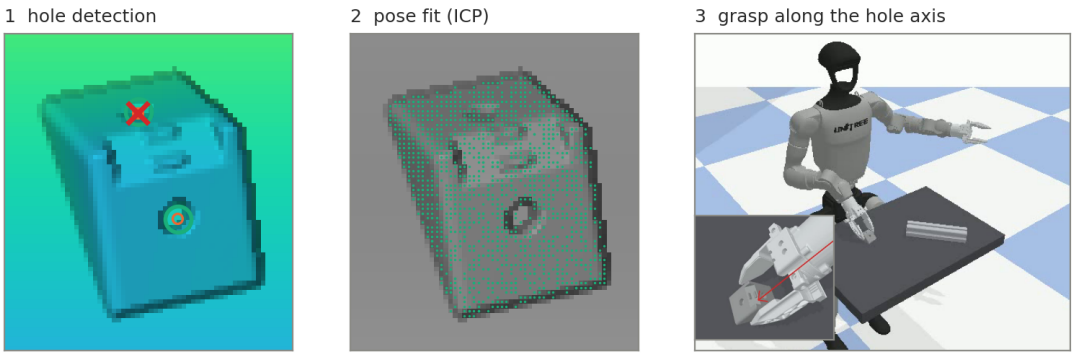


Figure 1: The three stages on ideal depth: hole detection, CAD fit and grasp (inset: approach along the detected hole axis).

2 Synthetic data pipeline and downstream tasks

2.1 Synthetic data

Scene. The parts are manufacturer CAD models from item (Line 8, 40×40). A profile cut to 200 mm and an angle bracket with its cover cap, simplified to free Ø8.5 mm holes. The table is sized to the G1's reach from an inverse-kinematics reachability map, which places the parts 0.47–0.68 m from the camera. Each random seed defines a scene where both parts lie in stable resting poses sampled by their probability, and the arms are raised to the sides so that the head camera sees the table unobstructed (Fig. 2).

Rendering and labels. Scenes are rendered by PyBullet (CPU). Each frame holds an RGB image, ideal z-depth (848×480, the D435's recommended depth resolution, $f = 447\,\text{px}$), an instance mask, and the camera intrinsics, camera pose and 6D part poses as ground truth. The camera model operates on saved frames and is therefore independent of the simulator.

Use of such synthetic data. The RGB-D camera in simulation is usable both for training a model and for evaluating a model-based system. In a learning pipeline, the depth passed through the camera model, together with the masks and 6D poses, would train a pose estimator, a segmentation network or a robot policy that will actually be deployed. Here the same frames serve as simulated inputs for evaluation of the developed code for the downstream tasks.

2.2 Downstream robot tasks

The robot has to pick the bracket by an accessible top hole and this is the feature a screw or pin goes through in assembly. The pipeline has three stages, each using only the output of the previous one.

(1) Hole detection. The detector removes the table plane, peels planar faces off the remaining clusters with RANSAC, and accepts a hole as a round gap of the right size enclosed by a face. The hole's centre is where the gap's pixel rays meet the face plane, its axis is the face normal, and the top hole is the one whose axis points most upward. The thresholds (points within 1 mm of a plane, 1.5 mm above the table) were set on ideal depth.

(2) Pose fit (ICP). The CAD fit recovers the full 6D pose of the bracket. Each upward hole candidate fixes five degrees of freedom; a search over the rotation about the hole axis scores how well the visible model surface matches the measured depth, and ICP refines the best match. All candidates are tried and the one with the best match is kept.

(3) Grasping. The Dex1 parallel jaws approach along the hole axis, close across the bracket's 40 mm width at an antipodal grasp point on the fitted CAD, and lift the part by 10 cm. Inverse kinematics plans a pre-grasp, a straight approach, the closing and the lift for either arm; a plan is rejected if the arm comes within 10 mm of the robot's own body anywhere along the motion, since PyBullet does not simulate self-collision. The plan is executed in PyBullet with physics. A grasp succeeds if the bracket rises above 50 mm with both jaws in contact after a one-second hold.

Why these stages. Each stage tests different levels of depth cue. Detection relies on a local cue, the hole is only a few pixels wide and is defined by a sharp depth edge, so it is the first thing the camera effects can destroy. The fit uses thousands of points over the whole visible surface; it should average out random noise but could not see a systematic error such as the depth bias, which would move the fitted part as a whole. The grasp tests whether the remaining error matters physically. The jaws should tolerate a few millimetres, so a perception error counts only if it makes the grasp fail. Chaining the stages, each fed only by the previous one, shows where a failure starts and whether it propagates to the robot's action.

A stand-in for a learned model. In a real cell this step could be a learned pose estimator or an end-to-end robot policy. We considered a model-based network (FoundationPose), but it needs a GPU stack and pretrained weights. The geometric pipeline is a simpler surrogate for it. It relies on the same depth cues a network would possibly use, whether for pose estimation or inside an end-to-end robot policy: planar faces, depth edges, small holes a few pixels wide and surface normals. If the camera model destroys or shifts these cues, a model that depends on them (as it was trained on such data) is likely to be affected as well. Its thresholds play a role similar to learned parameters, tuned on ideal synthetic depth, they resemble the weights of a network trained only on ideal synthetic data and then deployed on the real sensor. Unlike a network, the pipeline is transparent, so failures can be traced to the camera effect that caused it. The limitation is that a network trained on noisy depth may learn to tolerate some of the effects, this pipeline therefore measures how sensitive the cues are, not how a particular network would perform.

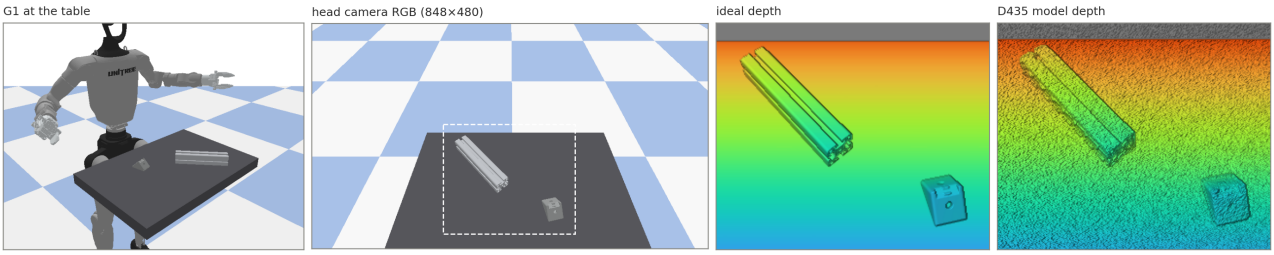


Figure 2: The G1 at the table, the head camera's RGB image, and ideal and D435-model depth of the marked region.

3 D435 camera model

Approach. We corrupt the ideal depth with closed-form models. A physics-based simulation of the infrared stereo pair would be more faithful, but the D4 matcher's parameters are undocumented and the simulation would itself need validation against real data. Learned noise models need real D435 depth of similar scenes, we found no released D435 noise generator. The scope rule is that every modelled effect has a published or reliable number. Effects with only heuristic numbers would not be traceable, so their impact could not be attributed to the real camera.

Effects included. The D435 measures disparity d between its two infrared imagers, with baseline $B = 50\,\text{mm}$, and reports depth $Z = fB/d$. The model has four parts: the output limits that every D435 frame obeys, and three effects applied in the order in which they arise in the camera, before the limits.

1. *Output limits* ("datasheet rules" in the experiment). Fixed properties of the sensor's output, independent of the scene: no depth in a band at the left image edge, which only the left imager sees; no depth closer than 195 mm at 848×480; and depth rounded to the default unit of 1 mm ([1], §4.5 and Table 4-12; [2], §8a). *Why:* every real frame would have them, the 1 mm rounding is of the order of the detector's thresholds.
2. *Edge fattening.* A stereo matcher compares windows of pixels; at a depth step the window would straddle both sides and the foreground tends to win, so the foreground would grow into the background and narrow gaps would close. No value is published for the D435. We use the only measured value, from Intel's previous R200 camera [3]: gaps narrower than 5 px merge, i.e. the foreground grows by 2 px. *Why:* the 8.5 mm hole is a depth step only 5–8 px wide at our distances (less when seen at an angle), the kind of gap a matching window would close.
3. *Axial noise*, added to the disparity of each pixel:

$$d' = d + s\,\xi, \quad \xi \sim \mathcal{N}(0,1) \qquad \Rightarrow \qquad \sigma_Z \approx \frac{Z^2}{fB}\,s, \tag{1}$$

   with $s = 0.08\,\text{px}$, the subpixel error of Intel's D435 noise curve [2], giving $\sigma_Z = 0.9\,\text{mm}$ at 0.5 m. *Why:* it is a documented effect, and at 0.5 m it is as large as the 1 mm tolerances that separate the bracket's faces from their surroundings.
4. *Depth bias* at the datasheet's accuracy limit of ±2 %: $Z' = (1 + b)\,Z$ with $b = \pm 0.02$, the sign drawn with equal probability. Since back-projection is linear in Z, every 3D point moves along its own viewing ray; shapes and orientations are preserved and the part shifts by about 10 mm. *Why:* it is a documented effect, and an offset of this size is comparable to the clearance of the jaws (54 mm open around the 40 mm bracket), so it can matter for the grasp even when perception looks correct.

Lateral error needs no separate term: in stereo the depth map lies on a fixed pixel grid, and we assume lateral error arises mainly from the axial noise through back-projection and from edge fattening at depth steps. Intel gives no lateral model; Ahn et al. measure the D435's lateral error at depth edges [4], which is consistent with this view, but the decomposition is ours.

Effects left out. Several effects should not matter for a static, indoor, single-frame, depth-only task: temporal noise, rolling shutter and motion blur (the depth imagers have a global shutter), ambient infrared, interference between cameras, defocus, colour-sensor effects and post-processing filters (off by default).

Dropouts (pixels without depth) are left out for lack of a usable number. The datasheet bounds them only on its test target: a fill rate of at least 99 % on a flat, matte white wall within 2 m [1], with no rule for where invalid pixels occur. At that level they would mostly be isolated pixels, which would fail the detector's size and

roundness tests; the real data in Section 5 suggest far more. No measurement exists for anodized aluminium; dropout patches on the profile, where its faces may reflect the projector pattern away at steep angles, would probably add false-hole candidates on the profile, which the CAD fit rejected in 99–100 % of draws in our sweep. The bracket is matte powder-coated, close to the datasheet target, and false depth on the profile's T-slots would likely be rejected the same way.

For the remaining effects no usable number exists: the dependence on incidence angle (the only published term [4] is hand-set and does not reproduce its own measurements), occlusion shadows inside the image, and the spatially correlated "egg carton" noise that Intel describes without a correlation length.

The noise level itself is uncertain. Ahn et al. [4] fitted the axial noise of a real D435 to about 2.7 times Intel's value at 0.5 m (about 0.2 px). Intel gives below 0.1 px on a well-textured target [2] and measured up to 0.32 px on a black glossy surface (on the D415) [5], so both levels are plausible. On the development scenes this level alone collapsed the pipeline (CAD fit in 4 of 150 draws, 15 scenes × 10 draws, against 143 with Intel's value).

4 Experiment: does the camera model change the downstream outcome?

The experiment asks whether the camera model changes the outcome of the three stages of Section 2: hole detection, CAD fit and grasp.

Protocol. The pipeline, the camera model and the hypotheses were developed on 15 scenes, which were then closed. The test protocol was fixed before any test run (git tag `final-experiment`). The 30 test scenes are the first seeds from 201 on for which every stage succeeds with ideal depth, with a collision-free arm motion, so that every failure is caused by the camera model. Seven depth conditions are compared: ideal, datasheet rules only, each effect alone, edge fattening with axial noise, and the full model. Each scene is run with 5 noise draws, the same in every condition, giving 150 draws per condition. Because the bias moves every position by about 10 mm, detection and pose count as correct within 15 mm; the strict 3 mm and 5 mm criteria are reported as accuracy.

Results. Table 1 and Fig. 3 summarise the sweep; per stage:

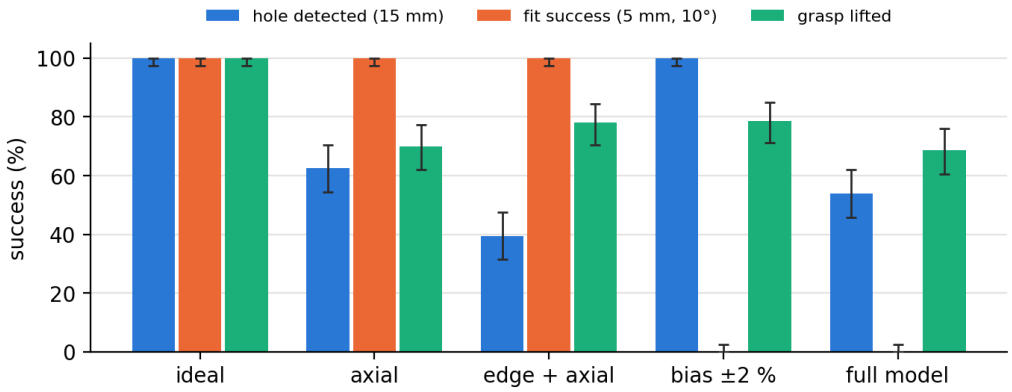


Figure 3: Success per stage with 95 % intervals; fit with the strict 5 mm criterion.

- *Hole detection* was affected by axial noise and edge fattening. Axial noise caused the largest single drop, to 63 %: it broke up the bracket's faces so that the real hole was no longer enclosed, and left gaps on the profile that passed as holes (Fig. 4, a development scene). Edge fattening alone did not prevent detection but tilted the hole axis (median axis error from 0.67° to 1.06°); with noise it lowered detection to 39 %, filling holes that noise alone left open (Fig. 5). Our edge model is likely too strong (Section 5), so this drop may be overstated. The bias and the rules left detection almost unchanged; the full model reached 54 %.

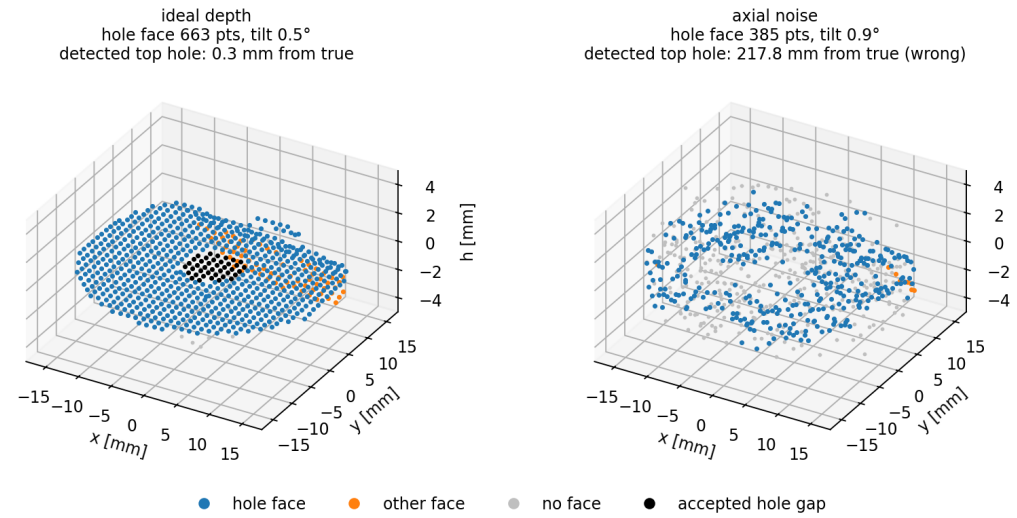


Figure 4: Depth points around the true top hole, ideal and with axial noise (black: the enclosed hole).

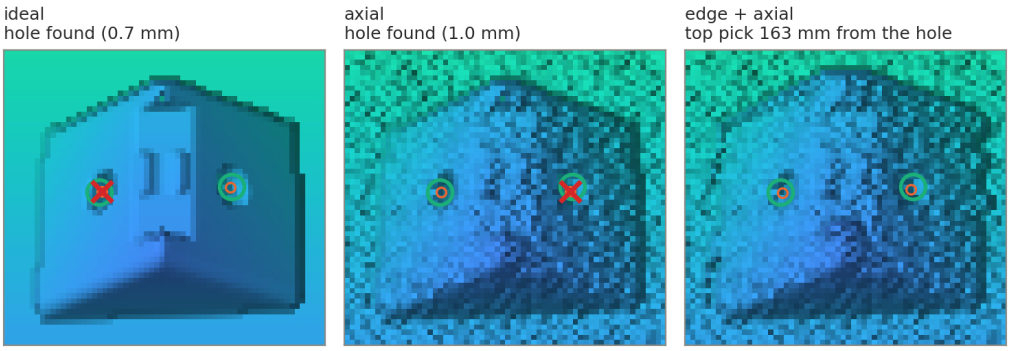


Figure 5: Hole detection on one noise draw. Green: true hole; red ×: the detector's pick.

- *CAD fit* was affected only by the depth bias, which shifted the whole pose by 10.7 mm along the viewing ray (11 mm under the full model). Noise and edge fattening did not stop it: by trying every candidate, the fit absorbed the detection errors and found the bracket in 99–100 % of draws.
- *Grasp* was affected by every camera condition: success fell from 100 % to 69–79 %, with the full model lowest (69 %). The failures happened at planning: the camera's pose errors moved the grasp target by a few millimetres, and the planner found no reachable, collision-free arm motion. Even the 1 mm rounding alone did this (77 %). Once planned, every grasp lifted the bracket; the open jaws (54 mm for the 40 mm bracket) absorbed the bias offset, though sometimes only by a corner, so that the bracket swung in the grip (Fig. 6). The size of the drop therefore reflects both the camera and the planner's sensitivity to small pose changes.

A repeat of the perception stages on Isaac Sim depth of the same scenes gave a similar pattern, within a few percentage points.

Condition	Hole detected	Pose found	Grasp lifted
ideal	100 %	100 %	100 %
datasheet rules	97 %	100 %	77 %
axial noise	63 %	100 %	70 %
edge fattening	100 %	100 %	73 %
edge + axial	39 %	100 %	78 %
depth bias ±2 %	100 %	100 % (10.7 mm off)	79 %
full model	54 %	99 % (11.1 mm off)	69 %

Table 1: Success per stage, 30 scenes × 5 draws; detection and pose within 15 mm.

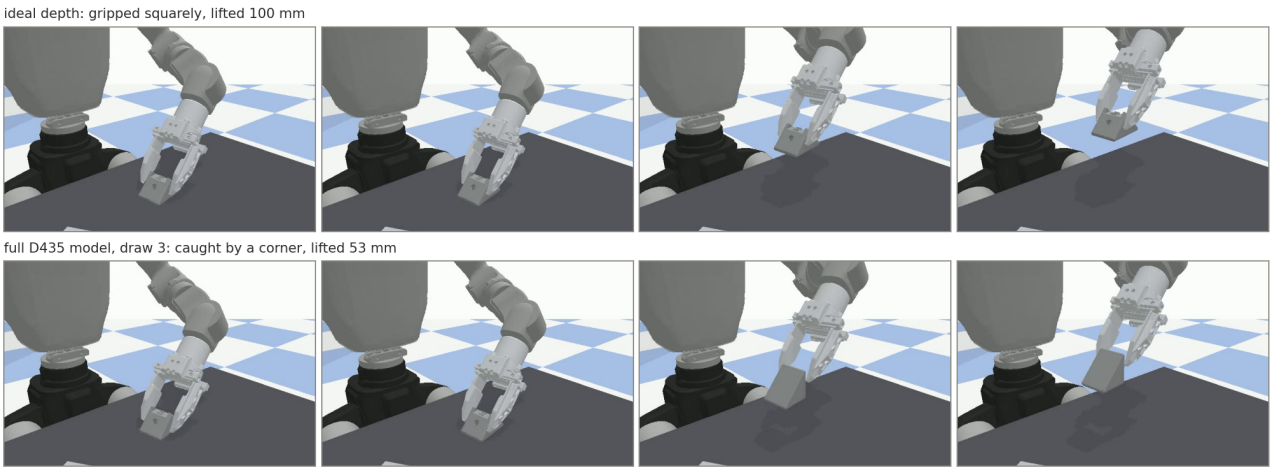


Figure 6: The same scene grasped with ideal depth (top) and the full camera model (bottom).

Implications for policies trained on such synthetic data. The two perception stages respond to the camera very differently. Hole detection depends on a local cue a few pixels wide and dropped from 100 % to 54 % under the full model. The CAD fit integrates over the whole visible surface and still found the bracket in 99 % of the cases. This suggests caution with training an end-to-end policy directly on depth images: such a policy would have to read the same local cues, the hole and the face edges, and would likely inherit their fragility, while its training data would also have to carry the right sensor effects. An explicit reconstruction step, model-based fitting such as ICP against CAD or any other module that recovers the object's state from the whole surface, absorbed most of the random sensor error in our pipeline, and a policy trained on that state may see less of the sim-to-real gap. It did not remove everything: a systematic error such as the depth bias passed through the fit as an 11 mm offset, which our jaws absorbed but a tighter grasp might not, and which moved the target the arm has to reach. We did not train a policy; this is an argument from the contrast between the two stages, not a measured result.

Mitigation. Adapting a model to the camera model is the analogue of training it on the camera model's data. We tried this for the detector on the 15 development scenes (5 draws each), scaling its thresholds with the modelled noise to $\max(\text{baseline}, k\,\sigma_Z)$. With axial noise alone, detection rose from 48 % to 100 % ($k = 2.5$). With edge fattening added, the same scaling closed holes and lowered detection (36 % to 16 %); only separate thresholds, a narrow face band and a raised object height, improved it (to 53 %). Under the full model no setting beat the baseline, and no threshold can remove the bias offset. Tuned and scored on the same development scenes, these numbers are optimistic.

5 How realistic is the camera model?

The model is built from Intel's documentation, not fitted to real data. As a basic check we applied it to the ground truth of HAMMER [6], which pairs real D435 depth with depth rendered from 3D scans of the same view (108 frames, 13 indoor scenes, 0.4–1.4 m; `camera_model/validate_hammer.py`, Fig. 7). The documented effects appeared with comparable size: the noise grew with distance as Intel's formula predicts, though about 1.7 times larger (2.0 against 1.2 mm plane residual at 0.53 m), and the real unit showed a scale bias of +3.1 %, somewhat above our ±2 % (part of it may come from the dataset's calibration). Two effects differed: the real noise appeared spatially correlated, and at depth edges the foreground bled in only partially (about a third of the pixels, another 40 % invalid) against almost all of them in our model, so our edge fattening is likely too strong. Dropouts (10–40 % of pixels on walls and the dark table), for which the documentation gives no number, are absent from the model. At this rate they would likely be an important effect; a value can be added once a calibration capture with the camera in the given scene is available. The check uses one unit and household objects, so it suggests the model is of the right kind and size, not that it predicts real success rates.

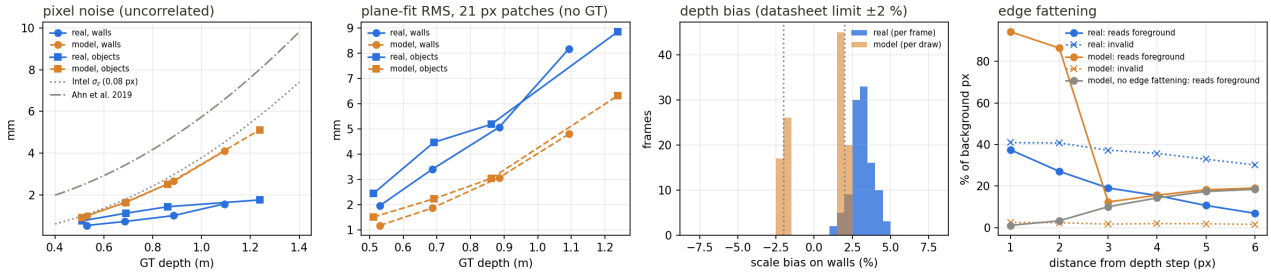


Figure 7: Real D435 depth (HAMMER, blue) and our model applied to its ground truth (orange).

6 What did not work

Ambiguity in the grasp result. The grasp drop mixes the camera's effect with the planner's brittleness. Even the 1 mm rounding alone failed 7 of the 30 scenes, the same scenes in every draw: it moved the estimated hole by under 1 mm, which switched the planner to another arm or grasp depth whose motion collided with the robot's body or had no inverse-kinematics solution. The camera causes these failures, but how many of them occur depends on the planner; a planner that tries several candidates would separate the two. The scene selection may add to this: test scenes were kept only if the grasp succeeded with ideal depth, which can favour scenes where the planner only just finds a plan and fails at the smallest change. The experiment therefore has scope for improvement on the grasp side.

What we could not validate. Without a camera or the parts, several steps rest on assumptions: the camera model was compared with real D435 depth only on household scenes (Section 5), not on anodized profiles and brackets; the edge-fattening width comes from an older Intel camera; dropouts on the metal parts are not modelled; the grasp physics (friction, contact) is PyBullet's and was not checked against a real gripper; and no learned model was trained, so the argument about learned policies (Section 4) is not tested.

Scope of the evaluation. The evaluation is synthetic: it shows how sensitive the pipeline is to the model. It covers one bracket, one profile and 30 scenes, so the rates carry intervals of about ±8 percentage points. How close the model is to a real D435 is checked in Section 5. The edge-fattening width comes from the R200, the noise is independent per pixel, there is no angle dependence, and PyBullet renders no material effects.

7 Outlook

Riskiest assumption. A perception module trained mainly on synthetic data learns the regularities of the renderer, not of the sensor since the ideal depth would have sharp edges, no noise, no missing pixels and no bias, and the model's parameters would adapt to exactly that. Our baseline detector suggested this effect: tuned on ideal depth, its detection fell from 100 % to 54 % under the modelled D435. A perception module trained or developed on ideal depth is poised to struggle in the same way, the more so when it relies on local cues, such as the hole and the face edges, which the camera degrades first. The ICP fit, which integrates over the whole surface, absorbed most of the random error but passed the systematic bias through as an 11 mm offset. The usual answer is domain randomisation, varying the sensor effects broadly enough that the real camera falls inside the range. It brings its own difficulties: the model has to cover variations the real sensor never produces, which costs data, capacity and possibly precision.

Whether as randomisation ranges or as a camera model, the sensor effects are rarely calibrated for the combination of camera and scene, and for the D435 several have no clear range: the edge-fattening width is known only for the older R200, no angle dependence is documented, and dropout is bounded only on a matte white wall (≤ 1 %), while real D435 depth on everyday surfaces is 10–40 % invalid. Even the documented values may not suit a specific case: the two published noise levels differ by a factor of 2.7, and the HAMMER unit showed a bias of +3.1 %, above the datasheet's ±2 %, and noise about 1.7 times larger and spatially correlated. A model adapted to the wrong value would meet the real camera as the ideal-tuned detector met the modelled one. The riskiest assumption here is therefore that the simulated sensor, its effects and their sizes, matches the real camera in the given scene where it matters for the task.

Additionally, real parts may differ from their CAD models: anodizing and powder coat vary in gloss, edges carry burrs and scratches, while our renderer shows ideal geometry with no material effects. Ambient infrared light in a factory hall, clutter around the parts and drift in the camera's mounting would add further deviations. We expect these to matter less than the sensor model, but none of them was tested.

First six months. The straightforward route is to calibrate the effects the model already has on the real camera in the real scene, turning the documented analytical model into a calibrated one: captures of the cell with the parts at known poses would fix the size of each modelled effect for this camera. The risk of this route is that it only fits the effects we chose to model, and other kinds of error that play a role in the scene would go unnoticed. If 3D scans of the cell serve as ground truth instead, their own error has to be small against the effects being calibrated, which at sub-millimetre noise is not guaranteed.

I would rather develop the camera model from camera views alone. Several views of the same parts, for example the head camera together with a camera on the robot's wrist, see each surface from different distances and angles. Merging them, for instance as a Bayesian update of an error model with the analytical model as its prior, could estimate the sensor's errors from the disagreement between views, and could also reveal effects that are not in the model. This is what I would pursue in the first six months, aiming at a camera model that is acceptable for the task without a ground-truth calibration.

In the longer run I would work on foundational perception modules, trained broadly, and on methods to fine-tune them to a given camera and scene with only a few views.

Use of AI tools

I used Claude Code throughout. It found, read and summarised the sources (datasheet, tuning guide, papers), looked up values and page references, and wrote and ran the code under my decisions. It drafted the report's prose from my notes and decisions; I revised and rewrote every section myself. I proofread the sources of the noise models: for each effect I checked the quoted passages, values and page references against the documents. I did not read the code of the task pipeline (scene, rendering, perception, grasping) line by line; I confirmed that its functions do what they should from their outputs. Claims were also checked against the source text, which found wrong page numbers, a citation attributed to the wrong camera (Keselman's values are for the R200) and a miscounted result. The main disagreement concerned edge fattening: the AI recommended modelling it with a swept width from the start and ranked it first priority multiple times. I declined, because no D435 value exists and an invented width would break traceability. In the end I tested it only with the R200's measured value and marked it as needing calibration.

References

[1] Intel, *Intel RealSense D400 Series Datasheet*, rev. 017, 2026.

[2] A. Grunnet-Jepsen, J. N. Sweetser, J. Woodfill, *Best-Known-Methods for Tuning Intel RealSense D400 Depth Cameras for Best Performance*, Intel, rev. 2.0.

[3] L. Keselman, J. I. Woodfill, A. Grunnet-Jepsen, A. Bhowmik, Intel RealSense Stereoscopic Depth Cameras, *CVPR Workshops*, 2017.

[4] M. S. Ahn, H. Chae, D. Noh, H. Nam, D. Hong, Analysis and Noise Modeling of the Intel RealSense D435 for Mobile Robots, *Int. Conf. on Ubiquitous Robots (UR)*, 2019.

[5] A. Grunnet-Jepsen, D. Tong, *Depth Post-Processing for Intel RealSense D400 Depth Cameras*, Intel, rev. 1.0.2.

[6] H. Jung, P. Ruhkamp, G. Zhai, N. Brasch, Y. Li, Y. Verdie, J. Song, Y. Zhou, A. Armagan, S. Ilic, A. Leonardis, N. Navab, B. Busam, On the Importance of Accurate Geometry Data for Dense 3D Vision Tasks, *CVPR*, pp. 780–791, 2023 (HAMMER dataset).